AI 桌面寵物・疊代一

透明視窗地基技術整理與心得

P0 環境建置 + P1-1 透明、置頂、點擊穿透的桌寵視窗（2026/11/5–11/6）



jelly・個人學習筆記・repo: weif3383-cloud/desktop-pet（dev 281d613）

疊代一做了什麼

11/5・規劃

企劃書 v0.1 → v0.6

回答 Q1–Q7、決定角色分遊走型／定點型、加入學習模式

11/6 上午・P0

環境建置

Unity 6.4（Universal 2D）、資料夾接上 GitHub dev、.gitignore

11/6 下午・P1-1

透明視窗地基

透明、置頂、點擊穿透、拖曳、不搶焦點、只開一隻

成果：桌面上一個**橘色暫代方塊**，背景完全透明，只有方塊可以點，其他地方點擊會穿透到桌面。
還沒做：狀態機（P1-2）、右鍵選單、存檔、Live2D 模型。

規劃階段的重要決策

主題	決定	為什麼
引擎	Unity 6.4 + C#（2D 專案）	動畫狀態機強；Live2D 本身是 2D，不需要 3D
角色	遊走型（小章魚）／定點型（人形）	人形在桌面走來走去很怪，小吉祥物才適合到處跑
主視窗	模式切換，同時只有一隻	Unity 只能有一個 OS 視窗
AI	Claude API，每人自備 key，可全關	不用架後端；關掉就是純桌寵、不花錢
模型	Live2D 最後才裝，先用方塊	先打地基，技術風險最高的透明視窗先驗證
版權	官方範例模型＋從本機資料夾載入	手遊拆包模型、別家吉祥物不能公開散佈

架構：三層，這次只蓋了最底層

Presentation（之後）
PetView、右鍵選單、聊天欄

Core 純 C#（之後）
PetBrain 狀態機、情緒、好感度

Platform ✓ 疊代一
Win32 透明／置頂／穿透、單一實例

Assets/_Project/Scripts/ 底下 6 個檔案

App/AppBootstrap：程式入口，自動建立物件、鎖 30fps

Platform/WindowController：決定「什麼時候」穿透

Platform/NativeWindow：負責「怎麼做」的 Windows 操作

Platform/Win32：DllImport 宣告

Platform/SingleInstance：只准一隻在跑

Debug/PlaceholderPet：暫代方塊（之後刪）

Unity 怎麼變成桌寵：四個步驟

1　背景透明

攝影機背景色 alpha = 0，再請 Windows 的 DWM 把整個視窗當「玻璃」：alpha=0 的地方就透出桌面

2　蓋滿螢幕＋置頂

視窗跟螢幕一樣大、HWND_TOPMOST 永遠在最上層，角色就能在整個桌面活動

3　動態點擊穿透

每幀檢查滑鼠下面有沒有角色：有就接收點擊，沒有就加上 WS_EX_TRANSPARENT 讓點擊穿過去

4　角色本體

跟一般 2D 遊戲一樣：Collider2D 判斷點擊、Animator＋狀態機控制動作（P1-2 才做）

另外要改的專案設定：關 DXGI Flip Model、關 URP HDR、繪圖 API 只留 Direct3D11、允許背景執行

程式碼①：NativeWindow 透明、置頂、穿透

```
// 讓攝影機背景 alpha=0 的地方透出桌面
var wholeWindow = new Win32.MARGINS { Left = -1 };
Win32.DwmExtendFrameIntoClientArea(_hwnd, ref wholeWindow);

// 永遠置頂（位置和大小不變）
Win32.SetWindowPos(_hwnd, Win32.HWND_TOPMOST, 0,0,0,0,
  SWP_NOMOVE | SWP_NOSIZE | SWP_FRAMECHANGED);

// 每個旗標是一顆開關，用 | 合在一起
uint style = Win32.WS_EX_LAYERED;     // 可透明
style |= Win32.WS_EX_NOACTIVATE;    // 不搶焦點
if (clickThrough)
  style |= Win32.WS_EX_TRANSPARENT; // 穿透
Win32.SetWindowLongPtr(_hwnd, GWL_EXSTYLE, (IntPtr)style);
```

hwnd：Windows 給每個視窗的編號，所有操作都要用它（GetActiveWindow 取得）

MARGINS Left=-1：-1 是特殊值＝整個視窗都延伸玻璃效果

DllImport：C# 直接呼叫 user32.dll／dwmapi.dll 的系統函式

64 位元要用 SetWindowLongPtr，不是 SetWindowLong

程式碼②：WindowController 每幀決定穿透

```
private void Update() {
  ReadCursor();
  Vector3 world =
_camera.ScreenToWorldPoint(CursorScreenPosition);
  IsOverInteractive = Physics2D.OverlapPoint(world) ≠ null;
#if !UNITY_EDITOR && UNITY_STANDALONE_WIN
  bool want = !IsOverInteractive && !HoldInteractive;
  // 狀態有變才通知 Windows，避免每幀呼叫
  if (want ≠ _window.IsClickThrough)
    _window.SetClickThrough(want);
#endif
}

// 穿透時 Unity 收不到滑鼠 → 直接問 Windows
Win32.GetCursorPos(out var p);
Win32.ScreenToClient(_hwnd, ref p);
return new Vector2(p.X, Screen.height - p.Y); // y 翻轉
```

HoldInteractive：拖曳中滑鼠暫時離開方塊也不能穿透，不然會拖到一半掉下來

GetAsyncKeyState：不管有沒有焦點都抓得到按鍵（最高位元 0x8000＝按著）

Y 軸翻轉：Windows 原點在左上，Unity 在左下

#if / #else：打包版問 Windows，編輯器走 Input System

程式碼③：程式入口與單一實例

AppBootstrap.cs

```
[RuntimeInitializeOnLoadMethod(
  RuntimeInitializeLoadType.AfterSceneLoad)]
static void Init() {
  if (!SingleInstance.TryAcquire()) {
    Application.Quit(); return;
  }
  Application.targetFrameRate = 30;
  new GameObject("WindowController")
    .AddComponent<WindowController>();
  // …建立 PlaceholderPet
}
```

用程式碼建物件，場景檔保持乾淨，Git 合併比較不會衝突

SingleInstance.cs

```
// 向 Windows 登記一個有名字的鎖
_handle = Win32.CreateMutexW(
  IntPtr.Zero, false,
  "Local\\DesktopPet_SingleInstance_7f3a");
bool alreadyExists =
  Marshal.GetLastWin32Error()
  == Win32.ERROR_ALREADY_EXISTS;
return _handle != IntPtr.Zero
      && !alreadyExists;
```

第二隻啟動時發現鎖已存在，就自己關掉

重構：「眼睛要瞎了」→ 拆出 NativeWindow

改之前：Start() 裡直接一大串 Win32

```
uint exStyle = Win32.WS_EX_LAYERED;
if (hideFromTaskbar) exStyle |= Win32.WS_EX_TOOLWINDOW;
Win32.SetWindowLongPtr(_hwnd, Win32.GWL_EXSTYLE, (IntPtr)exStyle);
Win32.SetWindowPos(_hwnd, Win32.HWND_TOPMOST, 0, 0, 0, 0,
    Win32.SWP_NOMOVE | Win32.SWP_NOSIZE | Win32.SWP_FRAMECHANGED);

SetClickThrough(true);
```

改之後：四行讀起來像目錄

```
_window = new NativeWindow(hideFromTaskbar);
_window.EnableTransparency();
_window.SetClickThrough(true);   // 一開始先穿透，滑鼠移到角色上才接收點擊
_window.MakeTopmost();
```

分工：WindowController 決定「什麼時候」做，NativeWindow 負責「怎麼」做

拆不拆的判斷：名字能說出程式碼說不出的意圖才值得拆；重複的「組樣式」抽成 BuildStyle()

註解規則：每個函式、屬性用 /// <summary>（等於 Python 的 ''' docstring）

issue.md：10 個問題，全部解決

#	現象	原因	類型
10	桌面出現兩個方塊	舊的沒關；Mono 的 Mutex 不是全系統共用	平台（只在打包版）
9	拖方塊時動態桌布暫停	點擊讓全螢幕視窗取得焦點	平台（只在打包版）
8	打包後整個螢幕全黑	Unity 6 自動選 DX12，不支援透明	平台（只在打包版）
7	找不到打包	打包在 Unity 裡，不是 VS Code	工具
6	CS0414 黃色警告	欄位只在 #if 打包區塊用到	工具
5	滑鼠移上去看不到說明	隱藏了 csproj；#if 區塊不分析	工具
4	看不到 Claude 改的內容	分頁開著未存檔的舊版	工具
3	資料夾很亂	VS Code 開錯資料夾	工具
2	git push 要帳號	git 還沒登入 GitHub	Git
1	.slnx 會被 commit	.gitignore 沒排除新格式	Git

ISSUE #8

打包後整個螢幕變全黑

原因

透明要靠舊式的畫面輸出，讓 Windows 把視窗後面的桌面透出來。DirectX 12 強制用新式「翻頁」輸出（Flip Model），整張畫面直接蓋上去，透明資訊就丟了。Unity 6 預設自動選 DX12。

解法

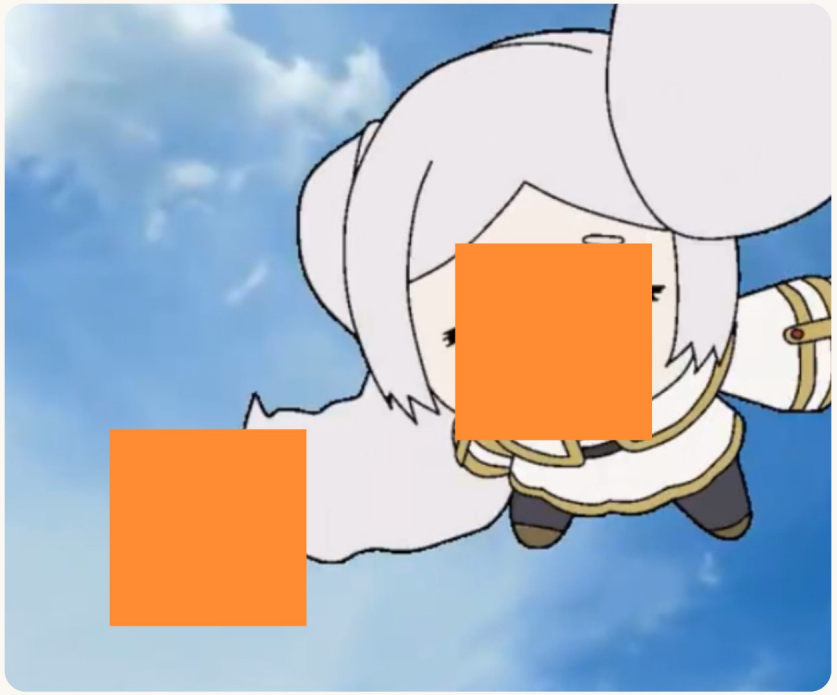
Project Settings → Player → Other Settings → Rendering：取消 **Auto Graphics API for Windows**，清單只留 **Direct3D11**。

脫困：Alt+F4 或 Ctrl+Shift+Esc 結束 DesktopPet

當時檢查過 Flip Model、HDR、攝影機 alpha 都正確，用「排除法」鎖定繪圖 API，一改就成功

ISSUE #9

拖動方塊時，動態桌布會暫停



現象：透明成功了（左圖），但只要點或拖方塊，動態桌布就停。

原因：點擊讓桌寵視窗取得「焦點」。它又蓋滿整個螢幕，桌布程式以為有全螢幕程式在使用，就暫停省電。

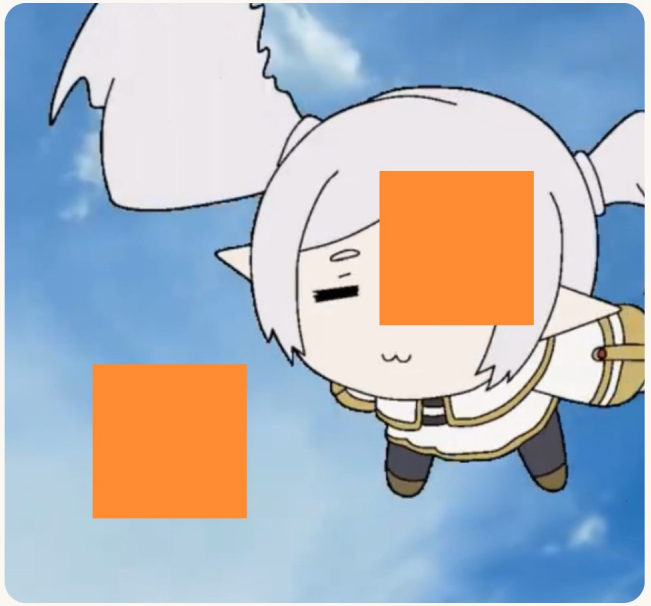
解法：視窗樣式加上 WS_EX_NOACTIVATE，被點也不變成使用中的視窗。

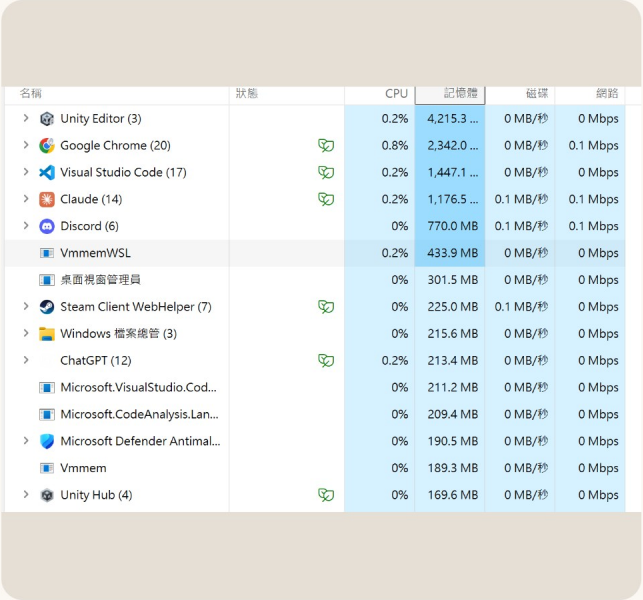
```
style ⊨ Win32.WS_EX_NOACTIVATE;
// = 0x08000000
```

後續注意：之後做聊天輸入框需要鍵盤，打字時要暫時允許取得焦點。

ISSUE #10

桌面上同時出現兩個方塊





名稱	狀態	CPU	記憶體	磁碟	網路
Unity Editor (3)		0.2%	4,215.3 ...	0 MB/秒	0 Mbps
Google Chrome (20)		0.8%	2,342.0 ...	0 MB/秒	0.1 Mbps
Visual Studio Code (17)		0.2%	1,447.1 ...	0 MB/秒	0 Mbps
Claude (14)		0.2%	1,176.5 ...	0.1 MB/秒	0.1 Mbps
Discord (6)		0%	770.0 MB	0.1 MB/秒	0.1 Mbps
VmmemWSL		0.2%	433.9 MB	0 MB/秒	0 Mbps
桌面視窗管理員		0%	301.5 MB	0 MB/秒	0 Mbps
Steam Client WebHelper (7)		0%	225.0 MB	0.1 MB/秒	0 Mbps
Windows 檔案總管 (3)		0%	215.6 MB	0 MB/秒	0 Mbps
ChatGPT (12)		0.2%	213.4 MB	0 MB/秒	0 Mbps
Microsoft.VisualStudio.Cod...		0%	211.2 MB	0 MB/秒	0 Mbps
Microsoft.CodeAnalysis.Lan...		0%	209.4 MB	0 MB/秒	0 Mbps
Microsoft Defender Antimal...		0%	190.5 MB	0 MB/秒	0 Mbps
Vmmem		0%	189.3 MB	0 MB/秒	0 Mbps
Unity Hub (4)		0%	169.6 MB	0 MB/秒	0 Mbps

推理：兩方塊能分開拖 → 不是同一程式的兩物件，而是**兩個 .exe**

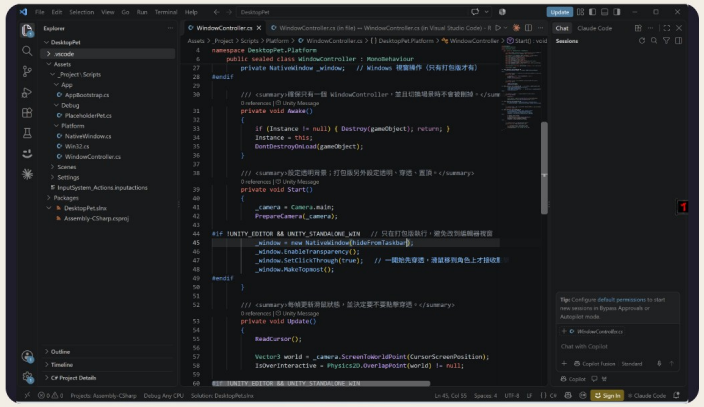
原因：Awake() 的「只留一個」只管同一程式；加上 #9 後 Alt+F4 關不掉舊的

第一版修正失敗：C# 的 System.Threading.Mutex 在 Unity Mono 裡不一定全系統共用

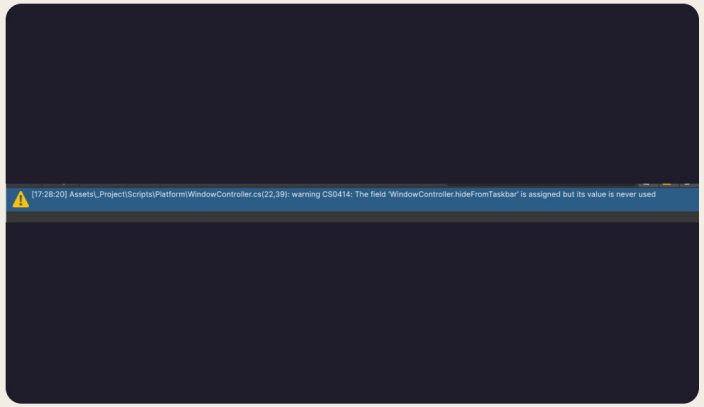
第二版：直接呼叫 Windows 的 CreateMutexW ✓

我的測試法：故意連開兩次 exe、亂點滑鼠。附帶發現：右鍵＝關閉太容易誤觸，之後換成右鍵選單。

工具類問題 #3–#7：VS Code 與 Unity 分工



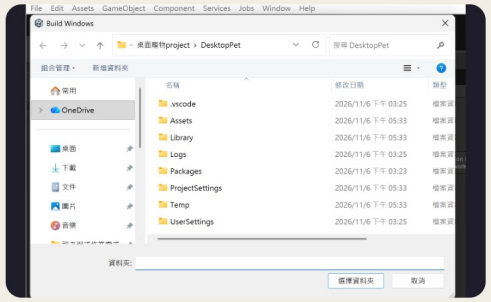
#5 看不到 /// 說明：settings.json 藏了 csproj，C# 擴充找不到專案；#if !UNITY_EDITOR 區塊本來就不分析



#6 CS0414 警告：hideFromTaskbar 只在打包區塊用到；用 #pragma warning disable 包住



#7 找不到打包：Build 在 Unity 的 File → Build Profiles；輸出選 DesktopPet/Builds





#3 資料夾很亂：看資源用 Unity 的 Project 視窗，VS Code 只寫程式。
#4 新內容不見：分頁有未存檔舊版時別按 Ctrl+S，用 Revert File。

Git 工作流程：從 PR 到直接改本機

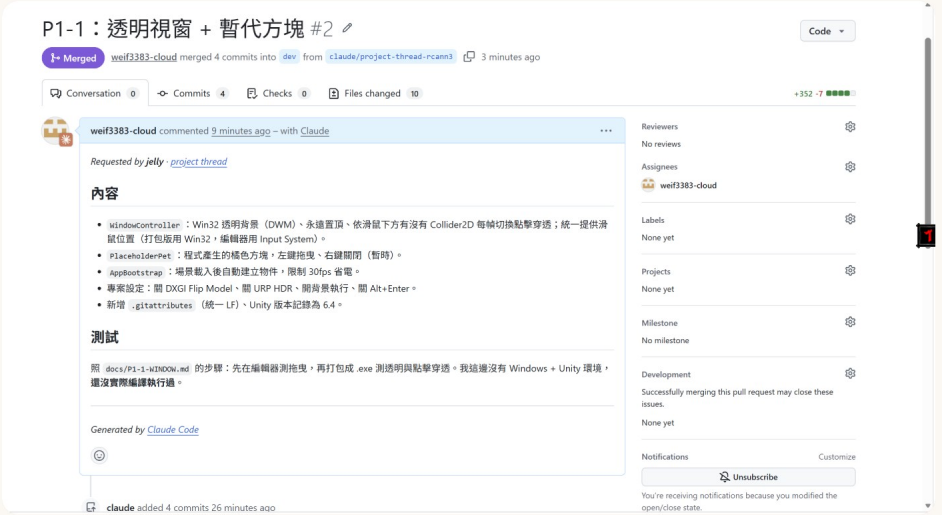
分支：main＝穩定版、dev＝開發主線、claude/...＝AI 草稿

一開始：Claude 推到自己的分支 → 開 PR → 我 Merge → 我 git pull

後來改成：Claude 直接改我電腦的資料夾，我在 VS Code「原始檔控制」看差異，自己決定 commit / push

.gitignore 保險：Library、Temp、Builds、*.slnx，還有 *.secret.json、secrets/（API key 外洩只能作廢重辦）

git 登入：credential.helper manager，瀏覽器登入一次就記住




```
+ # 機密：API Key 等絕對不能進 Git
+ *.secret.json
+ secrets/
```

心得

平台問題只能打包實測

三個最難的 bug 在編輯器裡完全測不出來。破壞性測試（狂點、連開兩次）抓到了兩個。

好讀比短重要

拆 class 讓 Start() 像目錄；每個函式都寫 ///；#if 區塊逐行加註解。

我的價值在判斷

知道要做什麼、把模糊需求講清楚、看出哪裡錯、真的做完。

AI 寫的程式要靠人驗證

Claude 沒有 Windows 跑不了程式；DX12、Mutex 的第一版都是我測出來才修對。

出 bug 是最好的學習時機

先猜原因 5 分鐘 → 看修改的那幾行 → 階段結束再讀全貌。

把問題記下來

issue.md 記現象、原因、解法，之後做學習網站就有素材。

下一步：疊代二

P1-2

角色狀態機

待機、走路、睡覺、點擊反應、被拖曳；學習模式第一個練習＋第一個測試

P1

右鍵選單＋存檔

取代「右鍵＝關閉」；JSON 存位置和名字

P1.5

換上 Live2D

官方範例模型 Hiyori、從本機資料夾載入

我的狀態機草稿：待機 → 點擊 → 喚醒 → 隨機走路／抓滑鼠；待機 → 過一段時間 → 睡覺 → 點擊 → 喚醒

待決定：平常會自己隨機活動，還是點擊後才開始動？動作做完都要回「待機」，拖曳優先度最高。